MULUNGUSHI UNIVERSITY
SCHOOL OF ENGEERING A ND TECHNOLOGY

STUDENT NAME: ABEL SIMASIKU

STUDENT ID: 202407015

COURSE: DATA STRUCTURES AND ALGORITHMS

LECTURE R: MR KALUBA

DUE DATE: 16/04/2026

ASSIGNMENT 3:

1. COMBINED JAVA PROGRAM FOR CONVERTING INFIX EXPRESSION TO PREFIX AND POSTFIX.

```
import java.util.Stack;

public class InfixConversion {

    static boolean isOperator(char c) {
        return (!Character.isLetterOrDigit(c));
    }

    static int getPrecedence(char c) {
        switch (c) {
            case '+':
            case '-':
                return 1;
            case '*':
            case '/':
                return 2;
            case '^':
                return 3;
        }
        return -1;
    }

    static String infixToPostfix(String exp) {
```

```
        StringBuilder result = new StringBuilder();
        Stack<Character> stack = new Stack<>();

        for (int i = 0; i < exp.length(); i++) {
            char c = exp.charAt(i);

                if (Character.isLetterOrDigit(c)) {
                result.append(c);
            }
                else if (c == '(') {
                stack.push(c);
            }
                else if (c == ')') {
                while (!stack.isEmpty() && stack.peek() != '(') {
                    result.append(stack.pop());
                }
                stack.pop();
            }
                else {
                while (!stack.isEmpty() && getPrecedence(c) <=
getPrecedence(stack.peek())) {
                    result.append(stack.pop());
                }
                stack.push(c);
            }
        }

          while (!stack.isEmpty()) {
            result.append(stack.pop());
        }

        return result.toString();
    }

        static String infixToPrefix(String exp) {
            StringBuilder input = new StringBuilder(exp);
        input.reverse();

          for (int i = 0; i < input.length(); i++) {
            if (input.charAt(i) == '(') {
                input.setCharAt(i, ')');
                i++;
            } else if (input.charAt(i) == ')') {
                input.setCharAt(i, '(');
                i++;
            }
        }
```

```
        String postfix = infixToPostfix(input.toString());

        StringBuilder prefix = new StringBuilder(postfix);
        prefix.reverse();

        return prefix.toString();
    }


    public static void main(String[] args) {
        String infix = "(A-B/C)*(A/K-L)";
        System.out.println("Infix Expression: " + infix);
        System.out.println("Postfix Expression: " + infixToPostfix(infix));
        System.out.println("Prefix Expression: " + infixToPrefix(infix));
    }
}
```

2. PSUEDO CODE FOR INFIX TO POSTFIX

```
FUNCTION infixToPostfix(expression):
    CREATE empty stack for operators
    CREATE empty string for result

    FOR each character c in expression:
        IF c is operand:
            ADD c to result
        ELSE IF c is '(':
            PUSH c onto stack
        ELSE IF c is ')':
            WHILE stack is not empty AND top of stack != '(':
                POP from stack and ADD to result
            POP '(' from stack
        ELSE IF c is operator:
            WHILE stack is not empty AND precedence(top of stack) >= precedence(c):
                POP from stack and ADD to result
            PUSH c onto stack

    WHILE stack is not empty:
        POP from stack and ADD to result
```

```
    RETURN result
```

3. PSUEDO CODE FOR INFIX TO PREFIX

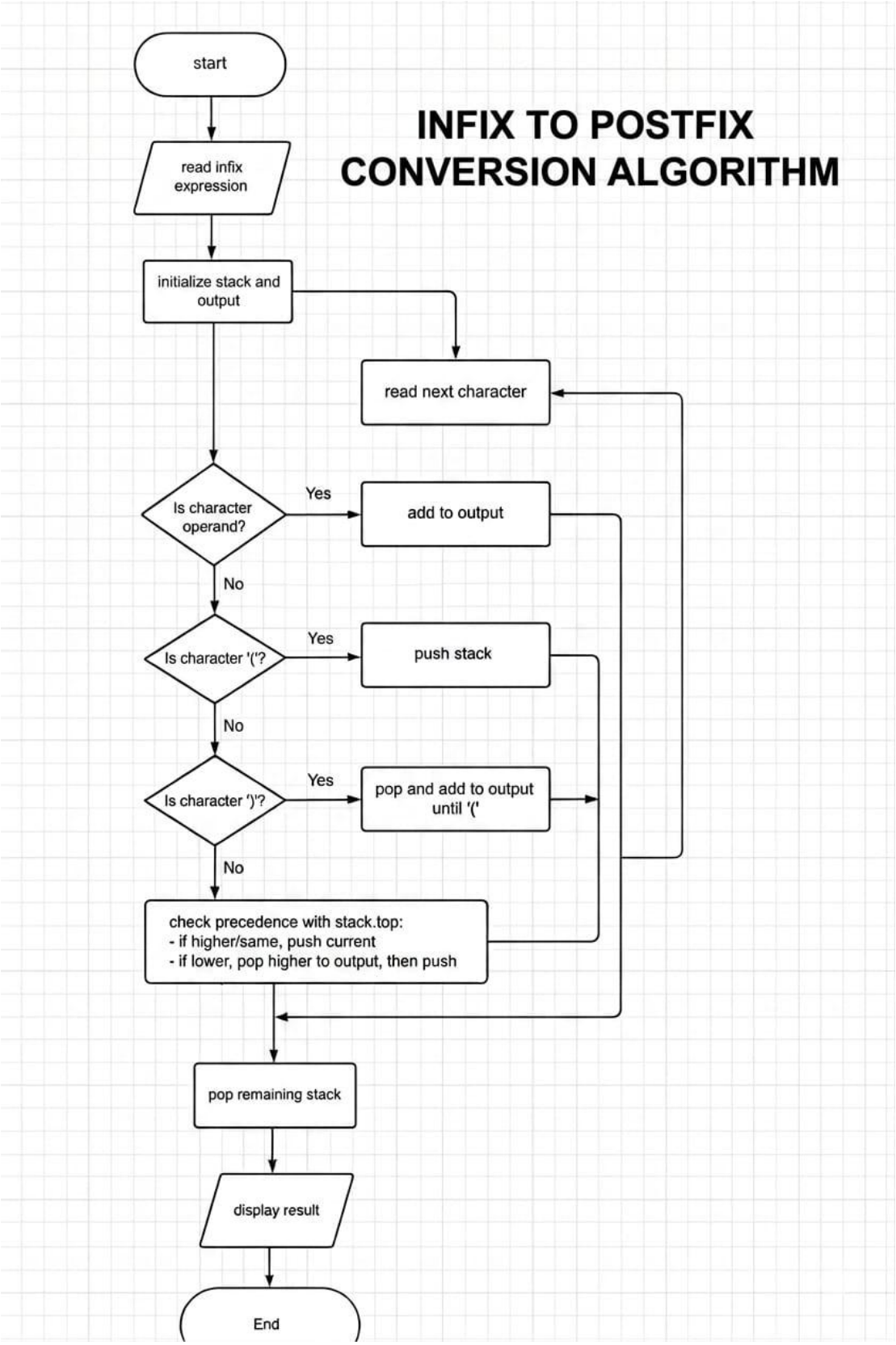
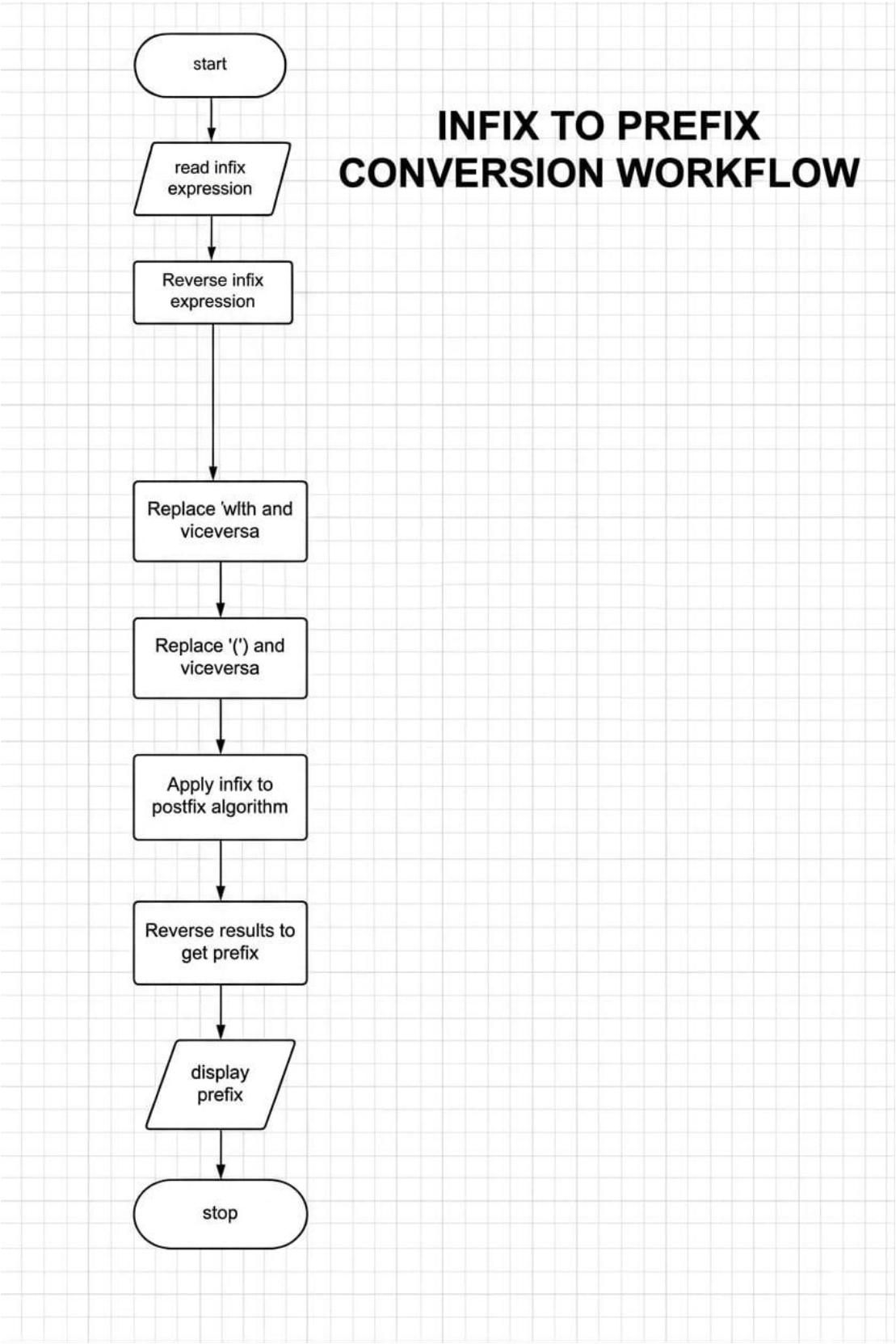
```
FUNCTION infixToPrefix(expression):
    REVERSE the expression
    SWAP '(' with ')' and vice versa

    postfix = infixToPostfix(reversed expression)

    prefix = REVERSE(postfix)

    RETURN prefix
```

4. WORKFLOW FOR CONVERTING INFIX TO POSTFIX



5. WORKFLOW FOR CONVERTING AN INFIX EXPRESSION TO PREFIX